

Assignment 2: OOP Foundations, Memory & Pass-by-Value (Practical)

Practical Coding Exam: 2 Object-Oriented Challenges (Easy & Medium)

Total Questions: 2 Practical	Difficulty: Easy & Medium	Format: Hands-on Source Code	Compiler: JDK 17+
------------------------------	---------------------------	------------------------------	-------------------

Submission & Execution Rules:

- All solutions must be compilable, working Java source files (.java) executing cleanly with javac and java.
- Validate user input, avoid unhandled runtime exceptions, and provide formatted console output as demonstrated.
- Each question must contain clear comments explaining the chosen algorithm and data structure.

[EASY LEVEL] — Encapsulated Bank Account with Strict Invariant Guards

PRACTICAL LAB

Problem Statement: Create an encapsulated class BankAccount.java and test harness BankApp.java enforcing data hiding and balance integrity.

Technical Requirements & Implementation Checklist:

- Fields: private final String accountNumber, private String accountHolder, private double balance.
- Constructors: Provide a 2-arg constructor (accountNumber, accountHolder) that chains to a 3-arg constructor (accountNumber, accountHolder, initialBalance) using this(...).
- Guarded Methods: deposit(double amount) (must be strictly > 0), withdraw(double amount) (cannot be negative, cannot exceed balance; deduct a \$2.00 fee if balance falls below \$100).
- Defensive Getters/Setters: Holder name cannot be null or empty; balance has no public setter.
- Write a test script in BankApp.java creating 2 distinct accounts, demonstrating independent Heap states, and proving invalid transactions are rejected.

Starter Code Template:

```
public class BankAccount {
    private final String accountNumber;
    private String accountHolder;
    private double balance;
    // TODO: Implement chained constructors, deposit, withdraw, and display
}
```

Sample Console Interaction:

Account ACC-101 created for Alice with \$500.00
Deposit \$200.00 -> New Balance: \$700.00
Withdraw \$650.00 -> Under minimum (\$100), fee \$2.00 applied -> New Balance: \$48.00
Withdraw \$100.00 -> FAILED: Insufficient funds (Available: \$48.00)

Evaluation Criteria: Encapsulation & constructor chaining | Balance invariant & withdrawal fee logic | Test harness verifying independent states

[MEDIUM LEVEL] — Smart Connected Vehicle Telemetry & Trip Computer

PRACTICAL LAB

Problem Statement: Implement VehicleTelemetry.java modeling a vehicle trip computer with overloaded constructors, state tracking, and fuel consumption modeling.

Technical Requirements & Implementation Checklist:

- Fields: vin (String), model (String), fuelCapacityLiters (double), currentFuelLiters (double), odometerKm (double), engineRunning (boolean).
- Overloaded Constructors with Chaining: Provide 3 constructors (basic with VIN/model defaulting to 50L capacity, full capacity, and pre-existing mileage), all chaining via this(...).
- Engine Lifecycle: startEngine() only if fuel > 0; stopEngine().
- Trip Execution: drive(double distanceKm, double speedKmh): Vehicle consumes 0.07L/km at normal speed (<= 90 km/h) and 0.09L/km at high speed (> 90 km/h). If fuel is exhausted mid-trip, calculate exact km traveled before stall.

- Write a simulation driver VehicleSimulator.java creating 2 vehicles, driving them on varied trips, and printing trip logs.

Starter Code Template:

```
public class VehicleTelemetry {  
    private final String vin;  
    private String model;  
    private double fuelCapacity;  
    private double currentFuel;  
    private double odometer;  
    private boolean engineRunning;  
    // TODO: Overloaded constructors chaining via this() and drive method  
}
```

Sample Console Interaction:

```
Vehicle VIN-9801 (Tesla/Civic) initialized. Fuel: 50.0L, Odo: 0.0km  
Engine started.  
Trip 1: Driving 200 km @ 80 km/h -> Consumed 14.0L fuel. Remaining: 36.0L, Odo: 200.0km  
Trip 2: Driving 500 km @ 110 km/h -> Out of fuel after 400.0 km! Engine stalled at Odo: 600.0km
```

Evaluation Criteria: Constructor chaining & field encapsulation | Fuel consumption & mid-trip stall calculation | Simulation driver script & formatted reports